Week_8_9_Li_Code_file_ipynb_final_version

May 2, 2026

1 1. Mount gdrive and import libs

[1]:
```
from google.colab import drive
drive.mount('/content/gdrive')
import pandas as pd
import numpy as np
import re
from pathlib import Path
from typing import Literal
import matplotlib.pyplot as plt

from skimage.filters import gaussian, threshold_otsu
from skimage.segmentation import *
import skimage.io as skio
from skimage.feature import graycomatrix, graycoprops
import skimage.morphology as skmor
import skimage.measure as skmea
from skimage.morphology import (
        disk,
        binary_opening,
        binary_closing,
        remove_small_objects,
        remove_small_holes
)

from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, Subset

!pip install xgboost
from sklearn.metrics import accuracy_score,classification_report,␣
 ↪confusion_matrix
from sklearn.preprocessing import LabelEncoder
from xgboost import XGBClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import StratifiedKFold,cross_validate,␣
↪GridSearchCV, ParameterGrid
```

```
Mounted at /content/gdrive
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-
packages (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in
/usr/local/lib/python3.12/dist-packages (from xgboost) (2.30.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages
(from xgboost) (1.16.3)
```

2 2.Unzip dataset zip file

[2]:
```
#!unzip "/content/gdrive/MyDrive/BMET5933/WEEK_8/hep2img.zip" -d "/content/
↪gdrive/MyDrive/BMET5933/WEEK_8"
#!unzip "/content/gdrive/MyDrive/BMET5933/WEEK_8/hep2img_large.zip" -d "/
↪content/gdrive/MyDrive/BMET5933/WEEK_8"
```

3 3.Dataset class definition:

3.1 AI acknowledge:

Chatgpt5.4 is used for learning how to heritate Dataset class and debugging errors for structure of this dataset class.

[3]:
```
class Hep2Dataset(Dataset):
        def __init__(self, dataset_root, csv_path, transform=None):
                """
                Initialize the dataset.

                Args:
                        dataset_root (str or Path): Root folder of the image␣
↪dataset.
                        csv_path (str or Path): CSV file containing image file␣
↪names and labels.
                        transform (callable, optional): Transform to apply to␣
↪each image.
                """
                self.dataset_root = Path(dataset_root)
                self.csv_path = Path(csv_path)
                self.transform = transform

                # Read all samples once and store them as a unified list
                self.samples = self.read_label(self.csv_path)
```

```
        def read_label(self, csv_path):
                """
                Read the CSV file and build a list of (image_path, label).

                Args:
                        csv_path (Path): Path to the CSV label file.

                Returns:
                        list: A list of tuples, where each tuple is␣
↪(image_path, label).
                """
                df = pd.read_csv(csv_path)

                samples = []
                for _, row in df.iterrows():
                        img_name = row["file"]
                        label = int(row["class"])  # Convert label to integer
                        mask_path = self.dataset_root / f'{str(img_name).
↪zfill(3)}_Mask.png'
                        img_path = self.dataset_root / f'{str(img_name).
↪zfill(3)}.png'
                        samples.append((img_path, mask_path, label))

                return samples

        def __len__(self):
                """
                Return the total number of samples in the dataset.
                """
                return len(self.samples)

        def __getitem__(self, idx):
                """
                Get one sample by index.

                Args:
                        idx (int): Index of the sample.

                Returns:
                        tuple: (image, mask, label)
                """
                img_path, mask_path, label = self.samples[idx]
                img = skio.imread(img_path)
                mask = skio.imread(mask_path)

                if self.transform is not None:
```

```
                        img = self.transform(img)

                return img, mask, label
```

4 4.Shape feature extractor:

Here I used circularity, solidity and eccentricity beacuse we don't have metadata like what we have in DICOM format. So some features like perimeter and area, we cannnot get.

4.1 AI acknowledge:

Chatgpt5.4 used for learning and designing shape feature extraction class, especially on method designing.

[4]:
```
class ShapeFeatureExtractor:
        def __init__(self):
                return

        def _get_largest_region(self, mask):
                """
                Get the largest connected component from a mask.

                Args:
                        mask (ndarray): Input mask.

                Returns:
                        regionprops object or None: Largest region if exists,␣
 ↪else None.
                """
                mask = mask > 0
                labeled_mask = skmea.label(mask)

                regions = skmea.regionprops(labeled_mask)
                if len(regions) == 0:
                        return None

                largest_region = max(regions, key=lambda x: x.area)
                return largest_region

        def _get_solidity_from_region(self, region):
                """
                Calculate solidity from a regionprops object.

                Args:
                        region: regionprops object or None.
                Returns:
```

```
                        float: Solidity value. Returns 0.0 if no valid region␣
↪exists.
                """
                if region is None:
                        return 0.0

                return float(region.solidity)

        def _get_eccentricity_from_region(self, region):
                """
                Calculate eccentricity from a regionprops object.

                Args:
                        region: regionprops object or None.
                Returns:
                        float: Eccentricity value. Returns 0.0 if no valid␣
↪region exists.
                """
                if region is None:
                        return 0.0

                return float(region.eccentricity)

        def _get_circularity_from_region(self, region):
                """
                Calculate circularity from a regionprops object.

                Formula:
                        circularity = 4 * pi * area / perimeter^2

                Args:
                        region: regionprops object or None.
                Returns:
                        float: Circularity value. Returns 0.0 if no valid␣
↪region exists
                                or perimeter is zero.
                """
                if region is None:
                        return 0.0

                area = region.area
                perimeter = region.perimeter

                if perimeter == 0:
                        return 0.0

                circularity = 4.0 * np.pi * area / (perimeter ** 2)
```

```
                return float(circularity)


        def get_shape_features(self, mask):
                """
                Extract all shape features from a single mask.

                Args:
                        mask (ndarray): Input mask.

                Returns:
                        dict: Dictionary containing shape features.
                """
                largest_region = self._get_largest_region(mask)
                single_feature = {
                        "circularity": self.
 ↪_get_circularity_from_region(largest_region),
                        "solidity": self.
 ↪_get_solidity_from_region(largest_region),
                        "eccentricity": self.
 ↪_get_eccentricity_from_region(largest_region),
                }
                return single_feature
```

5 5.GLCM feature exatractor:

[5]:
```
class GlcmFeatureExtractor:
        def __init__(self):
                """
                Initialize the GLCM feature extractor.

                Args:
                        image (ndarray): Input image.
                        label (int): Corresponding label.
                        mask (ndarray): Corresponding mask.
                """

                # glcm configuration
                self.distance = 1
                self.angle = 0 # angle for GLCM
                self.levels = 8 # number of gray levels for GLCM
                self.symmetric = False
                self.normed = False

        def get_quantized_image(self, image):
                """
```

```
                Get the gray-level image.

                Args:
                        image (ndarray): Input image.
                Returns:
                        ndarray: Gray-level image.
                """
                quantized_gray_image = np.floor(image / 256 * self.levels).
↪astype(np.uint8)
                return quantized_gray_image

        def get_masked_gray_image(self, image, mask):
                """
                Get the gray-level image with the background masked out.

                Args:
                        image (ndarray): Input image.
                        mask (ndarray): Input mask.
                Returns:
                        ndarray: Gray-level image with background masked out.
                """
                background_mask  = ~(mask > 0)
                gray_image = self.get_quantized_image(image)

                # only keep foreground region
                masked_image = gray_image.copy()
                masked_image[background_mask] = 0
                return masked_image

        def get_glcm_features(self, image, mask):
                glcm_features = {}

                # get masked gray image first
                masked_image = self.get_masked_gray_image(image, mask)

                # get GLCM matrix
                glcm = graycomatrix(
                                        masked_image.astype(np.uint8),
                                        distances=[self.distance],
                                        angles=[self.angle],
                                        levels=self.levels,
                                        symmetric=self.symmetric,
                                        normed=self.normed
                                        )
                # extract GLCM features
                glcm_contrast = graycoprops(glcm, prop='contrast')[0, 0]
```

```
                glcm_dissimilarity = graycoprops(glcm, prop='dissimilarity')[0,␣
↪0]
                glcm_homogeneity = graycoprops(glcm, prop='homogeneity')[0, 0]
                glcm_energy = graycoprops(glcm, prop='energy')[0, 0]
                glcm_correlation = graycoprops(glcm, prop='correlation')[0, 0]

                # store features in a dictionary
                glcm_features = {
                        "contrast": glcm_contrast,
                        "dissimilarity": glcm_dissimilarity,
                        "homogeneity": glcm_homogeneity,
                        "energy": glcm_energy,
                        "correlation": glcm_correlation
                }

                return glcm_features
```

6 6.Gray intensity feature extractor:

[6]:
```
class ColorFeatureExtractor:
        def __init__(self):
                return

        def get_masked_gray_image(self, image, mask):
                """
                Get the grayscale image with the background masked out.
                """
                background_mask = ~(mask > 0)
                masked_image = image.copy()
                masked_image[background_mask] = 0
                return masked_image

        def get_gray_max_min(self, image, mask):
                """
                Calculate the max and min pixel intensities in the masked image.
                """
                masked_image = self.get_masked_gray_image(image, mask)
                max_gray = masked_image[mask > 0].max()
                min_gray = masked_image[mask > 0].min()
                return max_gray, min_gray

        def get_gray_avg(self, image, mask):
                """
                Calculate the average pixel intensity in the masked image.
                """
                masked_image = self.get_masked_gray_image(image, mask)
```

```
                avg_gray = masked_image[mask > 0].mean()
                return avg_gray

        def get_gray_std(self, image, mask):
                """
                Calculate the standard deviation of pixel intensities in the␣
↪masked image.
                """
                masked_image = self.get_masked_gray_image(image, mask)
                std_gray = masked_image[mask > 0].std()
                return std_gray

        def get_gray_histogram(self, image, mask):
                """
                Calculate the grayscale histogram of pixel intensities in the␣
↪masked image.
                """
                masked_image = self.get_masked_gray_image(image, mask)
                histogram, bin_edges = np.histogram(masked_image[mask > 0],␣
↪bins=256, range=(0, 256))
                return histogram, bin_edges

        def get_gray_intensity_features(self, image, mask):
                """
                Calculate a set of grayscale intensity features for a masked␣
↪image.
                """
                gray_features = {}
                gray_features["avg_intensity"] = self.get_gray_avg(image, mask)
                gray_features["std_intensity"] = self.get_gray_std(image, mask)
                gray_features["max_intensity"], gray_features["min_intensity"]␣
↪= self.get_gray_max_min(image, mask)
                return gray_features
```

7 7.Featrue extraction process and Dataset initialization

[7]:
```
# Dataset initialization
dataset = Hep2Dataset(
        dataset_root="/content/gdrive/MyDrive/BMET5933/WEEK_8/hep2img_large",
        csv_path="/content/gdrive/MyDrive/BMET5933/WEEK_8/hep_img_large_gt.csv"
)
```

[8]:
```
# instantiate feature/ color extractors
shape_extractor = ShapeFeatureExtractor()
color_extractor = ColorFeatureExtractor()
```

```
glcm_extractor = GlcmFeatureExtractor()

feature_list = []
for i in range(len(dataset)):
        image, mask, label = dataset[i] # Unpack the tuple
        feature = {}
        # store features in a dictionary, and update the dictionary with␣
 ↪different types of features
        feature.update(shape_extractor.get_shape_features(mask))
        feature.update(color_extractor.get_gray_intensity_features(image, mask))
        feature.update(glcm_extractor.get_glcm_features(image, mask))
        feature['label'] = label # Use the unpacked label
        feature_list.append(feature)

# convert into dataframe and extract x and y for training
feature_df = pd.DataFrame(feature_list)
x = feature_df.drop(columns=['label'])
y = feature_df['label']
```

7.1 Feature extraction and Train/test split explanation

1. In this part, I converted each cell image into one row of numerical features. I used the mask to focus on the cell region instead of the background. The shape features describe the cell boundary and region, the gray intensity features describe the brightness distribution inside the cell, and the GLCM features describe texture patterns. After extracting these features, I stored them in feature_df, where each row represents one image sample and the label column is the class that the model needs to predict.

2. I used an 80/20 train/test split, which is a machine learning experience. This gives most of the data to the model for training, while still keeping a separate test set to evaluate how well the model works on unseen images. I also used stratify=y_encoded because this is a multi-class dataset, and I want the training and test sets to keep a similar class distribution. The random_state=42 makes the split reproducible, so the result can be checked again with the same data split.

7.2 AI acknowledge:

chatgpt5.5 used for learning and suggesting how to introduce this stratify train_test_spliting.

[9]:
```
# label converting
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# split the dataset into training and testing subsets, ratio is a␣
 ↪experience-based choice
train_test_split_ratio = 0.2
x_train, x_test, y_train,y_test = train_test_split(
                                                                    x,
```

```
                                                                                y_encoded,
                                                                                test_size␣
↪=
                                                                                train_test_spl
                                                                                random_state=4
                                                                                stratify=y_enc
                                                                                )
```

8 8.Classifier training

8.1 Evaluation interpretation

The XGBoost model achieved a test accuracy of about 0.8462, which means it correctly classified around 84.62% of the test images. I also used the classification report because accuracy alone does not show how each class performs. Precision shows how many predicted samples of a class are actually correct, recall shows how many real samples of a class are found by the model, and F1-score balances precision and recall. The macro average treats every class equally, while the weighted average considers the number of samples in each class. The confusion matrix shows the detailed prediction errors: values on the diagonal are correct predictions, and off-diagonal values are misclassified samples.

8.2 Why I chose XGBoost

I chose XGBoost as the basic classifier because my extracted image features are tabular numerical data, which tree-based models usually handle well. XGBoost builds many decision trees step by step. Each new tree tries to correct the mistakes made by the previous trees, so the final model can learn more complex relationships between shape, intensity, texture features and the image classes. It is also suitable here because the relationship between the features and the HEp-2 staining patterns is probably non-linear.

[10]:
```python
def evaluate_model(model_name, model, x_train, x_test, y_train, y_test):
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)

    accuracy = accuracy_score(y_test, y_pred)
    report_dict = classification_report(y_test, y_pred, output_dict=True)

    print(f"{model_name} Accuracy: {accuracy:.4f}")
    print("Classification Report:")
    print(classification_report(
        y_test,
        y_pred,
        target_names=[str(c) for c in label_encoder.classes_]
    ))

    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
```

```
    return {
        "model": model_name,
        "accuracy": accuracy,
        "macro_f1": report_dict["macro avg"]["f1-score"],
        "weighted_f1": report_dict["weighted avg"]["f1-score"]
    }
```

[11]:
```
# Model definition
xgb_model = XGBClassifier(
    n_estimators=500,
    max_depth=5,
    learning_rate=0.1,
    objective='multi:softmax',
    eval_metric='merror',
    random_state=42
)

xgb_results = evaluate_model("XGBoost", xgb_model, x_train, x_test, y_train,␣
 ↪y_test)
```

```
XGBoost Accuracy: 0.8462
Classification Report:
              precision    recall  f1-score   support

           1       0.81      0.72      0.76        18
           2       0.80      0.94      0.86        17
           3       0.86      0.90      0.88        20
           4       0.80      0.80      0.80        20
           5       1.00      0.88      0.93        16

    accuracy                           0.85        91
   macro avg       0.85      0.85      0.85        91
weighted avg       0.85      0.85      0.85        91

Confusion Matrix:
[[13  1  2  2  0]
 [ 1 16  0  0  0]
 [ 2  0 18  0  0]
 [ 0  3  1 16  0]
 [ 0  0  0  2 14]]
```

9 9.Challenge part:

9.1 Challenge 1 result discussion

In this challenge, I compared XGBoost, Random Forest, and SVM with an RBF kernel using the same extracted feature set. Based on the result table, XGBoost achieved the best performance with an accuracy of about 0.8462, followed by SVM RBF with about 0.7802 and Random Forest with about 0.7692. XGBoost also had the highest macro F1 and weighted F1 scores, so I identified it as the best-performing algorithm in this comparison.

XGBoost may perform better because this task uses tabular numerical features extracted from images, including shape, gray intensity, and GLCM texture features. These features may have non-linear relationships with the HEp-2 image classes. XGBoost is a boosting-based tree ensemble, so it builds trees sequentially and each new tree tries to correct errors made by the previous trees. This can help it model difficult class boundaries more effectively.

Random Forest is also a tree-based ensemble, but its trees are trained independently and combined by voting. This makes it stable, but it does not correct previous mistakes in the same way as XGBoost. SVM with an RBF kernel can also handle non-linear boundaries, but it depends more strongly on feature scaling and how well the classes are separated in the feature space. In this dataset, the boosted tree approach appears to fit the extracted image features better than the other two methods.

9.2 AI acknowledge:

chatgpt5.5 used for learning knowledge of Random forest and SVM; and used for polishing explaination.

9.3 Challenge 1: comparison between 3 models

[12]:
```
# model initialization, parameters are experience-based choices
# random forest
rf_model = RandomForestClassifier(
        n_estimators=500,
        max_depth=5,
        random_state=42,
    class_weight='balanced'  # Add class_weight to handle class imbalance
)

# SVM with RBF kernel, parameters are experience-based choices
svm_model= Pipeline([
        ('scaler', StandardScaler()),
        ('svc', SVC(kernel='rbf', random_state=42))
])
```

[13]:
```
# model evaluation and comparison
results = []
```

```
results.append(evaluate_model("XGBoost", xgb_model, x_train, x_test, y_train,␣
 ↪y_test))
results.append(evaluate_model("Random Forest", rf_model, x_train, x_test,␣
 ↪y_train, y_test))
results.append(evaluate_model("SVM RBF", svm_model, x_train, x_test, y_train,␣
 ↪y_test))
```

```
XGBoost Accuracy: 0.8462
Classification Report:
              precision    recall  f1-score   support

           1       0.81      0.72      0.76        18
           2       0.80      0.94      0.86        17
           3       0.86      0.90      0.88        20
           4       0.80      0.80      0.80        20
           5       1.00      0.88      0.93        16

    accuracy                           0.85        91
   macro avg       0.85      0.85      0.85        91
weighted avg       0.85      0.85      0.85        91

Confusion Matrix:
[[13  1  2  2  0]
 [ 1 16  0  0  0]
 [ 2  0 18  0  0]
 [ 0  3  1 16  0]
 [ 0  0  0  2 14]]
Random Forest Accuracy: 0.7692
Classification Report:
              precision    recall  f1-score   support

           1       0.65      0.83      0.73        18
           2       0.93      0.76      0.84        17
           3       0.78      0.90      0.84        20
           4       0.65      0.65      0.65        20
           5       1.00      0.69      0.81        16

    accuracy                           0.77        91
   macro avg       0.80      0.77      0.77        91
weighted avg       0.79      0.77      0.77        91

Confusion Matrix:
[[15  0  2  1  0]
 [ 2 13  0  2  0]
 [ 2  0 18  0  0]
 [ 4  1  2 13  0]
 [ 0  0  1  4 11]]
```

```
SVM RBF Accuracy: 0.7802
Classification Report:
              precision    recall  f1-score   support

           1       0.75      0.83      0.79        18
           2       0.87      0.76      0.81        17
           3       0.73      0.80      0.76        20
           4       0.70      0.80      0.74        20
           5       1.00      0.69      0.81        16

    accuracy                           0.78        91
   macro avg       0.81      0.78      0.78        91
weighted avg       0.80      0.78      0.78        91

Confusion Matrix:
[[15  1  1  1  0]
 [ 2 13  1  1  0]
 [ 2  0 16  2  0]
 [ 1  1  2 16  0]
 [ 0  0  2  3 11]]
```

[14]:
```
# show comparison results in a dataframe format
comparison_df = pd.DataFrame(results)
comparison_df
```

[14]:

	model	accuracy	macro_f1	weighted_f1
0	XGBoost	0.846154	0.848191	0.845733
1	Random Forest	0.769231	0.774488	0.771539
2	SVM RBF	0.780220	0.784576	0.782218

10 Challenge 2: 3 more features(same features with different other 3 angels(pi/4, pi/2, 3pi/4))

10.1 Challenge 2 result discussion

For the extended feature set, I kept the original shape features, gray intensity features, and GLCM texture features, then added extra GLCM texture features from three additional directions: 45 degrees, 90 degrees, and 135 degrees. The original GLCM feature extraction only used one angle, so the extended version gives the model more directional texture information from the masked cell region.

After training XGBoost with the extended GLCM feature set, the test accuracy remained 0.8462, which is the same as the original XGBoost model. Therefore, the extra features did not improve the overall prediction accuracy in this single train/test split. However, the confusion matrix and some class-level scores changed slightly. For example, class 3 improved in precision, while class 5 had a lower precision compared with the original result. This suggests that the added GLCM directions changed some individual predictions, but the total number of correct predictions stayed almost the same.

A possible reason is that the new GLCM features are related to the original texture features, so they may not provide completely new information for XGBoost.

10.2 AI acknowledge:

Chatgpt5.5 was used to learn and construct child classes 'ExtendedGlcmFeatureExtractor' from GlcmFeatureExtractor; also used for suggesting which extra features can be added.

[15]:
```
class ExtendedGlcmFeatureExtractor(GlcmFeatureExtractor):
        def __init__(self):
                super().__init__()
                # overwrite angles into 0, 45, 90, 135 degrees
                self.angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

        def get_glcm_features(self, image, mask):
                # initialize a dictionary to store GLCM features for multiple␣
↪angles
                glcm_features = {}

                # get masked gray image first
                masked_image = self.get_masked_gray_image(image, mask)

                # get GLCM matrix for multiple angles
                glcm = graycomatrix(
                                        masked_image,
                                        distances=[self.distance],
                                        angles=self.angles,
                                        levels=self.levels,
                                        symmetric=self.symmetric,
                                        normed=self.normed
                                        )

                # extract GLCM features for each angle and average them
                glcm_contrasts = graycoprops(glcm, prop='contrast')
                glcm_dissimilarities = graycoprops(glcm, prop='dissimilarity')
                glcm_homogeneities = graycoprops(glcm, prop='homogeneity')
                glcm_energies = graycoprops(glcm, prop='energy')
                glcm_correlations = graycoprops(glcm, prop='correlation')

                # store features in a dictionary with angle-specific keys
                for i in range(len(self.angles)):
                        glcm_features[f"contrast_angle_{i}"] =␣
↪glcm_contrasts[0, i]
                        glcm_features[f"dissimilarity_angle_{i}"] =␣
↪glcm_dissimilarities[0, i]
                        glcm_features[f"homogeneity_angle_{i}"] =␣
↪glcm_homogeneities[0, i]
                        glcm_features[f"energy_angle_{i}"] = glcm_energies[0, i]
```

```
                        glcm_features[f"correlation_angle_{i}"] =␣
 ↪glcm_correlations[0, i]

                return glcm_features
```

[16]:
```
# creaete instances of feature extractors
extended_glcm_extractor = ExtendedGlcmFeatureExtractor()
color_extractor = ColorFeatureExtractor()
shape_extractor = ShapeFeatureExtractor()

feature_list = []
for i in range(len(dataset)):
        image, mask, label = dataset[i] # Unpack the tuple
        # store shape, color, and texture features in a dict
        features = {}
        features.update(shape_extractor.get_shape_features(mask))
        features.update(color_extractor.get_gray_intensity_features(image,␣
 ↪mask))
        features.update(extended_glcm_extractor.get_glcm_features(image, mask))
        features['label'] = label # Use the unpacked label
        feature_list.append(features)

# convert into dataframe and extract x and y for training
new_feature_df = pd.DataFrame(feature_list)
x_extended = new_feature_df.drop(columns=['label'])
y_extended = new_feature_df['label']
```

[17]:
```
# encode labels
y_encoded = label_encoder.fit_transform(y_extended)

# split the dataset into training and testing subsets, ratio is a␣
 ↪experience-based choice
train_test_split_ratio = 0.2
x_extended_train, x_extended_test, y_extended_train, y_extended_test =␣
 ↪train_test_split(
                                                                x_extended,
                                                                y_encoded,
                                                                test_size␣

 ↪= train_test_split_ratio,

                                                                random_state=4
                                                                stratify=y_enc
                                                                )
```

[18]:
```
# train and evaluate the model with extended GLCM features
xgb_extended_results = evaluate_model("XGBoost with Extended GLCM", xgb_model,␣
 ↪x_extended_train, x_extended_test, y_extended_train, y_extended_test)
```

```
XGBoost with Extended GLCM Accuracy: 0.8462
Classification Report:
              precision    recall  f1-score   support

           1       0.81      0.72      0.76        18
           2       0.80      0.94      0.86        17
           3       0.90      0.90      0.90        20
           4       0.80      0.80      0.80        20
           5       0.93      0.88      0.90        16

    accuracy                           0.85        91
   macro avg       0.85      0.85      0.85        91
weighted avg       0.85      0.85      0.85        91

Confusion Matrix:
[[13  1  2  2  0]
 [ 1 16  0  0  0]
 [ 1  0 18  0  1]
 [ 1  3  0 16  0]
 [ 0  0  0  2 14]]
```

11 Challenge 3: Cross - validation

11.1 Challenge 3 result discussion

In this challenge, I repeated the model comparison using 5-fold stratified cross-validation. Instead of using only one fixed train/test split, cross-validation splits the dataset into five folds and trains/evaluates each model five times. Each fold is used as the test set once, while the other folds are used for training. I used stratified cross-validation because this is a multi-class problem, so each fold should keep a similar class distribution.

The cross-validation results show that XGBoost achieved the best average performance, with an average accuracy of about 0.8566 and a macro F1 score of about 0.8591. Random Forest and SVM RBF had lower average accuracies, both around 0.7417. This is consistent with the earlier single train/test split result, where XGBoost was also the best-performing classifier.

I think cross-validation gives a more reliable estimate than a single train/test split because the final result does not depend on only one random split of the data. In a small image dataset, one train/test split may be easier or harder depending on which samples are placed in the test set. Cross-validation reduces this problem by testing the model on different subsets of the data and reporting the average performance.

11.2 AI Acknowledge:

chatgpt5.5 used for learning how to add cross validation and polishing explaination.

[19]:
```python
# training and evaluation for comparison models

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
# evaluate the original XGBoost model with cross-validation(with extended GLCM features)
xgb_cv_results = cross_validate(
        xgb_model,
        x,
        y_encoded,
        cv=cv,
        scoring=['accuracy', 'f1_macro', 'f1_weighted']
)

# evaluate comparison models with cross-validation
# random forest
rf_cv_results = cross_validate(
        rf_model,
        x,
        y_encoded,
        cv=cv,
        scoring=['accuracy', 'f1_macro', 'f1_weighted']
)

# SVM with RBF kernel
svm_cv_results = cross_validate(
        svm_model,
        x,
        y_encoded,
        cv=cv,
        scoring=['accuracy', 'f1_macro', 'f1_weighted']
)
```

[20]:
```
cv_results = []
# store cross-validation results in a list of dictionaries
cv_results.append({
    "model": "XGBoost",
    "accuracy_mean": xgb_cv_results["test_accuracy"].mean(),
    "accuracy_std": xgb_cv_results["test_accuracy"].std(),
    "macro_f1_mean": xgb_cv_results["test_f1_macro"].mean(),
    "macro_f1_std": xgb_cv_results["test_f1_macro"].std(),
    "weighted_f1_mean": xgb_cv_results["test_f1_weighted"].mean(),
    "weighted_f1_std": xgb_cv_results["test_f1_weighted"].std()
})

cv_results.append({
    "model": "Random Forest",
    "accuracy_mean": rf_cv_results["test_accuracy"].mean(),
    "accuracy_std": rf_cv_results["test_accuracy"].std(),
    "macro_f1_mean": rf_cv_results["test_f1_macro"].mean(),
    "macro_f1_std": rf_cv_results["test_f1_macro"].std(),
```

```
    "weighted_f1_mean": rf_cv_results["test_f1_weighted"].mean(),
    "weighted_f1_std": rf_cv_results["test_f1_weighted"].std()
})

cv_results.append({
    "model": "SVM RBF",
    "accuracy_mean": svm_cv_results["test_accuracy"].mean(),
    "accuracy_std": svm_cv_results["test_accuracy"].std(),
    "macro_f1_mean": svm_cv_results["test_f1_macro"].mean(),
    "macro_f1_std": svm_cv_results["test_f1_macro"].std(),
    "weighted_f1_mean": svm_cv_results["test_f1_weighted"].mean(),
    "weighted_f1_std": svm_cv_results["test_f1_weighted"].std()
})

# convert cross-validation results into a DataFrame for better visualization
cv_results_df = pd.DataFrame(cv_results)
cv_results_df
```

[20]:

	model	accuracy_mean	accuracy_std	macro_f1_mean	macro_f1_std	weighted_f1_mean	weighted_f1_std
0	XGBoost	0.856606	0.039851	0.859084	0.039521	0.857149	0.039090
1	Random Forest	0.741685	0.071394	0.749160	0.069991	0.745882	0.069483
2	SVM RBF	0.741661	0.041999	0.747641	0.045282	0.743544	0.042681

12 Challenge 4: Parameter grid fine tune

12.1 Challenge 4 explanation

In this challenge, I used grid-search cross-validation to tune the XGBoost hyperparameters. The parameter grid contains several possible values for n_estimators, max_depth, and learning_rate. Grid search tests the Cartesian product of these values, meaning that every possible combination is evaluated using cross-validation. This is more systematic than manual tuning because manual tuning may only test a few combinations and can easily miss a better setting.

For this experiment, I used accuracy as the scoring metric because the previous model comparisons in this notebook mainly used accuracy. Keeping the same metric makes the grid-search result easier to compare with the earlier XGBoost results. The best parameters printed by grid_search.best_params_ are the XGBoost settings that achieved the highest average cross-validation accuracy among the tested combinations. The value printed by grid_search.best_score_ is the best mean cross-validation accuracy.

12.2 AI acknowledge:

chatgpt5.5 used for polishing explaination.

13

[21]:
```
param_grid = {
    "n_estimators": [100, 300, 500],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.01, 0.05, 0.1]
}

xgb_grid_model = XGBClassifier(
    objective="multi:softmax",
    eval_metric="merror",
    random_state=42
)

grid_search = GridSearchCV(
    estimator=xgb_grid_model,
    param_grid=param_grid,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1
)

grid_search.fit(x_extended, y_encoded)

print("Best parameters:")
print(grid_search.best_params_)

print("Best CV accuracy:")
print(grid_search.best_score_)
```

```
Best parameters:
{'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 100}
Best CV accuracy:
0.8631990231990232
```

[ ]:
```
# These are for exporting the notebook as a PDF later if needed
!apt-get update
!sudo apt-get install texlive-xetex texlive-fonts-recommended␣
 ↪texlive-plain-generic pandoc
!jupyter nbconvert --to pdf /content/gdrive/MyDrive/BMET5933/WEEK_8/
 ↪Week_8_9_Li_Code_file_ipynb_final_version.ipynb
```

```
 0% [Working]              Hit:1 https://cloud.r-project.org/bin/linux/ubuntu
jammy-cran40/ InRelease
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
```

```
Hit:6 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:7 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Ign:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Ign:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Ign:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Ign:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Ign:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
0% [Working]
```